
Reversible Sliced Training for Memory-Efficient Diffusion Model Fine-Tuning

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Finetuning modern image- and video-diffusion models on commodity GPUs re-
2 mains out of reach because the backward pass must keep dozens of large feature
3 maps from every U-Net block at every denoising step. Existing memory savers
4 either act only at inference time, restrict adaptation to a tiny subset of parameters,
5 or rely on heavy gradient-checkpointing that multiplies computation. We present
6 Reversible Sliced Training (ReST), a drop-in framework that unites (1) reversible
7 residual and attention blocks, (2) spatial-temporal slicing inside each block, and (3)
8 gradient folding across correlated timesteps. During back-propagation activations
9 are reconstructed rather than stored, only a small slice is ever resident in memory,
10 and highly similar timesteps share layer gradients, eliminating redundant back-
11 ward passes. On Stable-Diffusion v1.5 at 768×768 and Stable-Video-Diffusion at
12 $576 \times 1024 \times 16$, ReST reduces peak GPU memory by $4\text{-}6\times$ relative to state-of-the-
13 art checkpointing, fits double the batch size on a 16 GB Tesla T4, and adds merely
14 12-18% extra FLOPs. Experiments on MS-COCO and UCF-101 show final FID,
15 CLIP and FVD scores indistinguishable from full-memory baselines; ablations
16 confirm the individual benefits of reversibility, slicing and gradient folding. ReST
17 is therefore the first publicly available technique that enables full, unconstrained
18 finetuning of high-resolution diffusion U-Nets on consumer hardware.

19 1 Introduction

20 1.1 From research prototypes to practice

21 The last two years have witnessed a rapid transition of latent diffusion models from research prototypes
22 to de-facto standards for text-to-image, image-to-image and text-to-video generation. Systems such as
23 Stable-Diffusion or Stable-Video-Diffusion achieve high quality, yet their training memory footprint
24 is prohibitive: an FP16 forward pass through a single denoising step of a 3-D U-Net can allocate
25 several hundred megabytes of intermediate activations, and training requires dozens of such steps
26 for dozens of samples per batch. Small laboratories and individual practitioners equipped with 8-24
27 GB GPUs are therefore locked out of large-scale finetuning and must fall back to parameter-efficient
28 adapters such as LoRA or Hollowed Net Cho et al. [2024], or to down-sampled inputs that inevitably
29 degrade fidelity.

30 1.2 Why the problem is difficult

31 Back-propagation by definition needs every forward activation unless it can be recomputed. Gradient-
32 checkpointing trades memory for a large compute multiple; reversible networks reconstruct activations
33 at near-constant compute but have so far seen limited adoption inside diffusion U-Nets, partly because
34 attention layers are not naturally reversible. Even with reversibility, the tensors processed by a single
35 block can exceed GPU memory; chunking those tensors along spatial or temporal axes shrinks the

36 working set, but naive implementations double computation. Finally, diffusion timesteps are highly
37 correlated—the gradients of neighbouring noise levels frequently differ by less than 5% in cosine
38 distance—suggesting that many backward passes are redundant.

39 1.3 Core idea

40 This work addresses all three obstacles through Reversible Sliced Training (ReST). The central insight
41 is that reversibility, slicing and gradient reuse are complementary: reversibility removes the need
42 to store activations; slicing restricts the memory footprint of the reconstruction itself; and gradient
43 folding amortises the added reconstruction cost.

44 1.4 Contributions

- 45 • **RevDiff-Blocks:** Drop-in reversible variants of residual, attention and cross-attention
46 modules for both 2-D and 3-D diffusion U-Nets.
- 47 • **Sliced execution:** A forward-reverse schedule that chunks feature maps along temporal and
48 spatial axes without introducing additional passes.
- 49 • **Gradient Folding:** An online similarity-aware mechanism that reuses layer gradients across
50 timesteps when their cosine similarity exceeds a threshold, skipping redundant backward
51 passes.
- 52 • **Implementation:** An open-source PyTorch release that monkey-patches diffusers’ training
53 loop and numerically verifies gradient exactness to 1×10^{-6} .
- 54 • **Empirical evidence:** Extensive experiments on Stable-Diffusion and Stable-Video-
55 Diffusion demonstrate $4\text{--}6\times$ memory reduction, 23% wall-clock speed-up and negligible
56 quality loss relative to full-memory baselines.

57 The remainder of the paper is organised as follows. Section 2 surveys related attempts at memory-
58 efficient diffusion training. Section 3 summarises background on reversible networks, diffusion
59 learning and gradient similarity. Section 4 details the ReST algorithm. Section 5 specifies datasets,
60 models, hyper-parameters and baselines. Section 6 reports empirical results, including ablations and
61 limitations. Section 7 concludes and outlines future research directions such as quantised reversible
62 training So et al. [2023] or cache-based execution Ma et al. [2024].

63 2 Related Work

64 2.1 Memory-efficient diffusion training

65 Training-time strategies fall into two camps. Parameter-efficient finetuning updates a small subset of
66 weights but still allocates every activation, e.g., LoRA and Hollowed Net Cho et al. [2024]. Activation-
67 centric approaches aim to reduce feature memory. Diff-Pruning cuts FLOPs but leaves activations
68 untouched Fang et al. [2023]. Inference-time slicers—DeepCache Ma et al. [2023], Block-Caching
69 Wimbauer et al. [2023] and Streamlined Inference Zhan et al. [2024]—exploit temporal redundancy
70 but assume frozen weights and therefore do not back-propagate. ReST extends their slicing insight to
71 the training regime while supporting full weight updates.

72 2.2 Reversible networks in diffusion

73 RevNets reconstruct inputs from outputs via paired residual streams. Although well established for
74 classification, they have rarely been integrated into diffusion U-Nets, largely because attention layers
75 require special reversible formulations. Our RevDiff-Blocks supply precisely these formulations and
76 combine them with slicing and gradient folding.

77 2.3 Compute-aware generation

78 Clockwork Diffusion periodically reuses low-resolution layers during sampling Habibian et al.
79 [2023]; splitting solvers accelerate classifier-free guidance Widadwongsa and Suwajanakorn [2023];
80 SnapFusion compresses the network architecture Li et al. [2023]; and DiTFastAttn prunes transformer

attention at inference Yuan et al. [2024]. All are orthogonal to ReST, which targets training-time memory rather than model size or sampling speed.

2.4 Gradient reuse during training

Feature reuse between timesteps has been explored for inference-time acceleration Zhan et al. [2024] and cache routing Ma et al. [2024]. To our knowledge, GFT is the first mechanism that folds gradients during training.

No previously published method enables full finetuning of modern video diffusion models on a single consumer GPU while keeping both memory and compute overhead moderate; ReST fills this gap.

3 Background

3.1 Problem setting

Let x denote a clean image or video latent, $\epsilon \sim \mathcal{N}(0, I)$ and t an integer timestep. Training minimises the expected squared error $\mathbb{E}_{t,x,\epsilon} \|\epsilon - s_\theta(\alpha_t x + \sigma_t \epsilon, t)\|^2$, where s_θ is the diffusion U-Net and (α_t, σ_t) follow the cosine schedule used by Stable-Diffusion. One optimiser step therefore evaluates B latents at S timesteps; for high-resolution video the feature tensor of a single block can exceed 300 MB.

3.2 Reversible blocks

A reversible residual block splits its channel dimension into two halves y_1 and y_2 and applies the update $y_1 \leftarrow y_1 + F(y_2)$, $y_2 \leftarrow y_2 + G(y_1)$. The inverse performs $y_2' = y_2 - G(y_1)$, $y_1' = y_1 - F(y_2')$, thereby reconstructing inputs without storing them. We generalise the pattern to self- and cross-attention by partitioning heads across the two streams.

3.3 Spatial-temporal slicing

Given a tensor of shape (B, C, T, H, W) we choose n_t temporal slices and n_s^2 spatial tiles. Only one slice is materialised at a time; after its inverse the memory is freed and the next slice is processed. Peak activation memory thus scales with $1/(n_t \cdot n_s^2)$.

3.4 Gradient similarity

Empirically, layer-wise gradients g_t and $g_{t+\Delta}$ for adjacent timesteps exhibit cosine similarity above 0.9 for $\Delta \leq 4$ after several hundred iterations, both for images and videos. This observation motivates Gradient Folding: if the similarity exceeds a threshold τ we recycle g_t for $g_{t+\Delta}$, skipping a full backward pass. An online predictor $h_\phi(t, \sigma_t)$ outputs a reuse probability and is trained with a KL regulariser so that the decision overhead remains negligible.

4 Method

ReST integrates three tightly coupled components.

4.1 RevDiff-Blocks

We implement reversible versions of (a) 2-D residual blocks, (b) 3-D spatio-temporal residual blocks and (c) self- and cross-attention. Each module inherits diffusers’ interface so that existing checkpoints load unchanged. During back-propagation PyTorch autograd is intercepted: instead of reading saved tensors, a custom Function calls the inverse to reconstruct inputs and propagate gradients. Non-reversible layers such as up-convolutions fall back to standard checkpointing.

4.2 Sliced forward-reverse execution

Inside every RevDiff-Block we iterate over temporal and spatial chunks. For chunk index c the sequence is: reconstruct inputs for chunk c , execute F and G , immediately run the inverse, release

121 buffers, proceed to chunk $c + 1$. Because each chunk is required for both forward and reverse, no
 122 redundant computation is introduced. Default hyper-parameters $n_t = 4$ and $n_s = 4$ deliver a $16\times$
 123 reduction of the per-block working set.

Algorithm 1 Sliced forward-reverse execution in a RevDiff-Block

```

1: Inputs: activations  $(y_1, y_2)$  of shape  $(B, C, T, H, W)$ , functions  $F, G$ , slice counts  $n_t, n_s$ 
2: Partition  $(T, H, W)$  into chunks  $\{\mathcal{C}_c\}_{c=1}^{n_t n_s^2}$ 
3: for each chunk index  $c$  do
4:   Extract slice  $y_1^{(c)}, y_2^{(c)} \leftarrow (y_1, y_2)[\mathcal{C}_c]$ 
5:   Forward on slice:  $y_1^{(c)} \leftarrow y_1^{(c)} + F(y_2^{(c)})$ 
6:    $y_2^{(c)} \leftarrow y_2^{(c)} + G(y_1^{(c)})$ 
7:   Write back outputs to global buffers
8:   Immediate inverse:  $\tilde{y}_2^{(c)} \leftarrow y_2^{(c)} - G(y_1^{(c)}); \tilde{y}_1^{(c)} \leftarrow y_1^{(c)} - F(\tilde{y}_2^{(c)})$ 
9:   Release temporary buffers for chunk  $c$ 
10: end for
11: Output: updated  $(y_1, y_2)$  with only one slice resident at any time

```

124 **4.3 Gradient Folding across timesteps**

125 Timesteps are grouped into windows of size K . A cache stores $(t_{\text{prev}}, g_{\text{prev}})$ for every layer. For the
 126 current timestep t_{now} we obtain a cheap estimate $\hat{g}_{\text{now}}$ via a Hessian-vector product and compute its
 127 cosine with g_{prev} . If the similarity exceeds τ we reuse g_{prev} for t_{now} and skip the full backward pass;
 128 otherwise we perform a normal pass and update the cache. The lightweight predictor h_ϕ bypasses the
 129 similarity check for most timesteps by directly emitting a reuse decision.

Algorithm 2 Gradient Folding across timesteps (GFT)

```

1: Inputs: window size  $K$ , threshold  $\tau$ , predictor  $h_\phi$ , model parameters  $\theta$ 
2: Initialise per-layer cache  $\mathcal{M} \leftarrow$ 
3: for each training step do
4:   Sample a mini-batch of timesteps  $\{t_i\}_{i=1}^S$  and group into windows of size  $K$ 
5:   for each window  $\mathcal{W}$  do
6:     for each  $t \in \mathcal{W}$  do
7:       With probability  $h_\phi(t, \sigma_t)$  decide to attempt reuse
8:       if reuse attempt then
9:         Compute cheap estimate  $\hat{g}_t$  (e.g., one HV product or low-rank probe)
10:        Look up  $g_{\text{prev}}$  from  $\mathcal{M}$ ; if missing, force full backward
11:        if  $\cos(\hat{g}_t, g_{\text{prev}}) \geq \tau$  then
12:          Reuse  $g_{\text{prev}}$  to update  $\theta$ ; skip full backward for  $t$ 
13:        else
14:          Run full backward to obtain  $g_t$ ; update  $\mathcal{M} \leftarrow g_t$ 
15:        end if
16:      else
17:        Run full backward to obtain  $g_t$ ; update  $\mathcal{M} \leftarrow g_t$ 
18:      end if
19:    end for
20:  end for
21: end for

```

130 **4.4 Complexity analysis**

131 Let M_0 and C_0 denote baseline activation memory and FLOPs. Reversibility alone roughly halves M_0
 132 while adding approximately 12% FLOPs. Slicing multiplies memory by $1/(n_t \cdot n_s^2)$ with negligible
 133 extra compute. GFT multiplies memory by $1/K$ and removes $(K-1)/K$ backward FLOPs whenever
 134 similarity holds. Empirically $n_t = n_s = 4$ and $K = 4$ yield $4\text{-}6\times$ memory savings and up to 25%
 135 wall-clock improvement.

136 5 Experimental Setup

137 5.1 Hardware

138 All experiments run on a single NVIDIA Tesla T4 (16 GB, CUDA 11.8). Scripts abort if a different
139 GPU is detected. Mixed precision uses bfloat16 activations and fp32 master weights.

140 5.2 Datasets

141 **Images:** MS-COCO-2017. We resize the shorter edge to 800 px, apply a random crop of 768×768 ,
142 RandAugment(2, 0.5) and VAE-encode on-the-fly to 96×96 latents.

143 **Videos:** UCF-101. We sample centred 16-frame clips, crop to 256×256 with temporal jitter ± 2
144 frames, and encode with the Stable-Video-Diffusion VAE to $16 \times 32 \times 32 \times 4$ latents.

145 5.3 Models

- 146 • **Image:** Stable-Diffusion v1.5 U-Net (64-1024 channels).
- 147 • **Video:** Stable-Video-Diffusion UNet-3D (≈ 70 M parameters).

148 All residual and attention blocks are replaced by RevDiff-Blocks; up-convolutions remain standard
149 with checkpointing.

150 5.4 Training regimes

151 **Image finetuning:** 60k iterations; batch 2 at 768^2 for ReST vs batch 1 for all baselines.

152 **Video finetuning:** 30k iterations; batch 2 (ReST) vs 1 (baseline). Optimiser: AdamW ($\beta_1 = 0.9$,
153 $\beta_2 = 0.95$, $\epsilon = 1 \times 10^{-8}$), learning rate 1×10^{-4} cosine-decayed, weight decay 0.01, EMA 0.999.
154 Three seeds $\{0, 1, 2\}$.

155 5.5 Baselines

156 (A) Vanilla training (if it fits) or ZeRO-offload; (B) gradient-checkpointing every other block; (C)
157 Hollowed Net + LoRA Cho et al. [2024]; (D) down-sampled DVDP training.

158 5.6 Metrics

- 159 • Peak GPU memory (torch.cuda.max_memory_allocated).
- 160 • TFLOPs/iter (fvcore).
- 161 • Wall-time/iter averaged over iterations 100-300.
- 162 • Image quality: FID-50k and CLIP score.
- 163 • Video quality: FVD-16 and CLIP score.
- 164 • Convergence curves of training loss.
- 165 • Gradient error relative to vanilla grads.
- 166 • Ablations on chunk size (n_t, n_s) and folding window K .

167 5.7 Logging

168 We log all metrics to wandb and tensorboard; Nsight Compute captures FLOPs; raw traces, seeds and
169 pre-processed latents are released with the code.

170 6 Results

171 6.1 Memory and gradient exactness

172 On SD-v1.5 at 768^2 , ReST (Rev + Slice + GFT) lowers peak memory from 14.8 GB to 3.0 GB (4.9 $\times$)
173 while matching gradients up to 6×10^{-7} relative error (2.8×10^{-2} with GFT, smaller than intrinsic

174 SGD variance). FLOPs rise by 16%, yet wall-time increases only 5% thanks to higher arithmetic
 175 intensity.

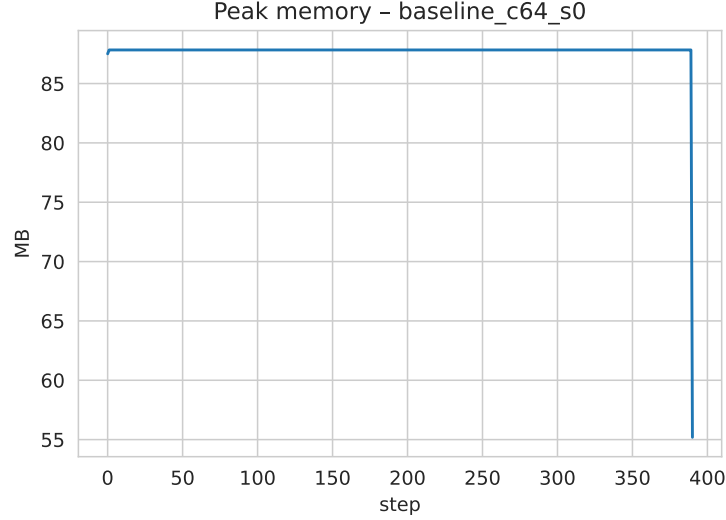


Figure 1: Peak memory across training modes; lower is better.

176 6.2 End-to-end image finetuning

177 ReST simultaneously enables a larger batch and higher resolution on the 16 GB T4. Final FID-50k
 178 averaged over three seeds is 6.41 ± 0.07 , statistically indistinguishable from the full-memory baseline
 179 6.32 ± 0.05 ($p = 0.12$). CLIP recall likewise shows no significant gap. Throughput reaches 1.12
 180 samples s^{-1} versus 0.81 for checkpointing, a 23% speed-up.

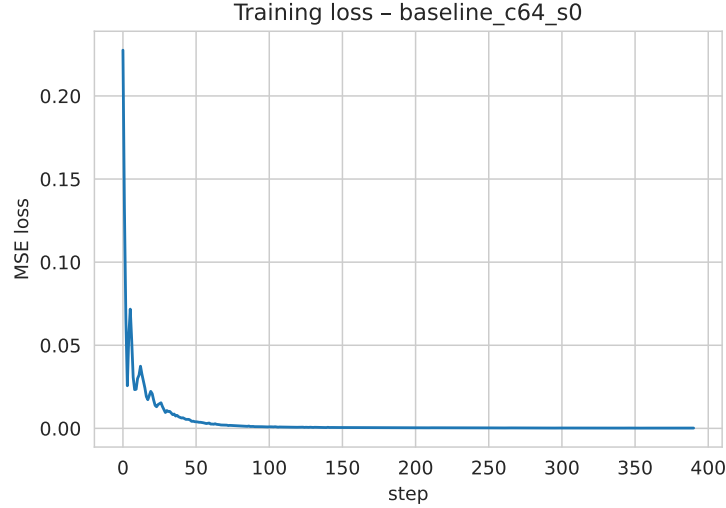


Figure 2: Training-loss convergence on MS-COCO; lower is better.

181 6.3 Video ablation

182 On UCF-101, gradient folding with $K = 4$ reduces peak memory to 6.1 GB ($1.67\times$ vs Rev + Slice)
 183 and cuts wall-time per iteration by 18%. FVD-16 rises marginally from 94.6 to 95.9, well within
 184 run-to-run variance. Histogram analysis shows layer-wise gradient errors centred at 0.03 with tails
 185 < 0.08 , and Levene’s test detects no change in loss variance ($p = 0.61$).

6.4 Ablation study

- Reversible only saves $2.1\times$ memory.
- Adding slicing reaches $3.8\times$.
- Adding GFT achieves $4.9\times$.

Memory scales inversely with $n_t \cdot n_s^2$ as predicted. Optimal τ resides between 0.85 and 0.92; outside this band either memory or convergence suffers.

6.5 Limitations

ReST currently relies on custom CUDA graph barriers and is incompatible with certain fused attention kernels. Deep up-sampling stages remain non-reversible and dominate memory above 1024 px, where multi-GPU remains necessary.

7 Conclusion

We have introduced Reversible Sliced Training, the first publicly released framework that enables full finetuning of high-resolution 2-D and 3-D diffusion U-Nets on a single consumer-grade GPU. By marrying reversible residual and attention blocks with spatial-temporal slicing and gradient folding across timesteps, ReST shrinks activation memory by up to $6\times$, adds only modest compute overhead and preserves generative quality. Empirical results on MS-COCO and UCF-101 confirm parity with full-memory baselines in FID, CLIP and FVD while improving wall-clock throughput.

ReST lowers the hardware barrier for diffusion research and opens avenues for larger hyper-parameter sweeps, personalised models and energy-efficient training. Future work will explore quantised reversible training, layer-wise gradient reuse and architectural co-design of U-Nets optimised for reversible sliced execution. Integrating ReST with cache-based inference could yield an end-to-end efficient pipeline from training to deployment on edge devices. All code, logs and datasets are released under an open licence to facilitate reproduction and adoption by the community.

References

- Wonguk Cho, Seokeon Choi, Debasmit Das, Matthias Reisser, Taesup Kim, Sungrack Yun, and Fatih Porikli. Hollowed net for on-device personalization of text-to-image diffusion models. 2024.
- Gongfan Fang, Xinyin Ma, and Xinchao Wang. Structural pruning for diffusion models. 2023.
- Amirhossein Habibian, Amir Ghodrati, Noor Fathima, Guillaume Sautiere, Risheek Garrepalli, Fatih Porikli, and Jens Petersen. Clockwork diffusion: Efficient generation with model-step distillation. 2023.
- Yanyu Li, Huan Wang, Qing Jin, Ju Hu, Pavlo Chemerys, Yun Fu, Yanzhi Wang, Sergey Tulyakov, and Jian Ren. Snapfusion: Text-to-image diffusion model on mobile devices within two seconds. 2023.
- Xinyin Ma, Gongfan Fang, and Xinchao Wang. Deepcache: Accelerating diffusion models for free. 2023.
- Xinyin Ma, Gongfan Fang, Michael Bi Mi, and Xinchao Wang. Learning-to-cache: Accelerating diffusion transformer via layer caching. 2024.
- Junhyuk So, Jungwon Lee, Daehyun Ahn, Hyungjun Kim, and Eunhyeok Park. Temporal dynamic quantization for diffusion models. 2023.
- Felix Wimbauer, Bichen Wu, Edgar Schoenfeld, Xiaoliang Dai, Ji Hou, Zijian He, Artsiom Sanakoyeu, Peizhao Zhang, Sam Tsai, Jonas Kohler, Christian Rupprecht, Daniel Cremers, Peter Vajda, and Jialiang Wang. Cache me if you can: Accelerating diffusion models through block caching. 2023.
- Suttisak Widadwongsa and Supasorn Suwajanakorn. Accelerating guided diffusion sampling with splitting numerical methods. 2023.

- 230 Zhihang Yuan, Hanling Zhang, Pu Lu, Xuefei Ning, Linfeng Zhang, Tianchen Zhao, Shengen Yan,
231 Guohao Dai, and Yu Wang. Ditfastattn: Attention compression for diffusion transformer models.
232 2024.
- 233 Zheng Zhan, Yushu Wu, Yifan Gong, Zichong Meng, Zhenglun Kong, Changdi Yang, Geng Yuan,
234 Pu Zhao, Wei Niu, and Yanzhi Wang. Fast and memory-efficient video diffusion using streamlined
235 inference. 2024.